PINAKI IT CONSULTANT
PRIVATE LIMITED

FREE SAMPLE
NOTES

PYTHON BASICS

A Beginner's Guide to Programming with Python

PINAKI IT CONSULTANT PRIVATE LIMITED

Welcome to your free sample of our Python notes. These pages are written for students who have never programmed before. Each topic is explained in simple language and followed by short, working code examples, their exact output, and a clear explanation of how the code works. By the end you will have written real programs and built a complete mini project.

WHAT YOU WILL LEARN

- Install Python and run your first program
- Syntax, comments and indentation
- Variables and data types
- Input and output
- Operators
- Decision making with if, elif and else
- Loops: for and while
- Strings, lists and tuples
- Functions and basic error handling
- A complete mini project

```
# Your very first Python program
print("Hello, World!")
>>> Hello, World!
```

Learn more at **www.pinakiithub.com**

1 Introduction to Python

What is Python?

Python is a free, general-purpose **programming language**. A programming language is a set of rules we use to give instructions to a computer. Python was designed so that its code reads almost like plain English, which makes it one of the easiest languages for a first-time programmer.

You write your instructions in a text file. A program called the **Python interpreter** then reads them line by line and carries them out.

Why is Python popular?

- **Easy to read and write:** short, clean syntax with few symbols.
- **Beginner friendly:** you can focus on logic instead of fighting the language.
- **Large community:** tutorials, answers and ready-made tools are easy to find.
- **Widely used in industry:** from small start-ups to large enterprises.

Key features of Python

- **Simple syntax** that is close to English.
- **Interpreted:** code runs line by line, so testing is quick.
- **Free and open source** to download and use.
- **Cross-platform:** Windows, macOS and Linux.
- **Rich libraries:** ready-made code for maths, data, web and AI.

Where is Python used?

Field	What Python is used for	Popular tools
Web development	Building websites and web services (APIs)	Django, Flask
Data analytics	Cleaning, analysing and presenting data	Pandas, Matplotlib
AI and machine learning	Building systems that learn patterns from data	scikit-learn, TensorFlow
Automation	Automating repetitive tasks such as renaming files or sending reports	Built-in modules, Selenium
Scientific computing	Numerical and scientific calculations	NumPy, SciPy

Python in action

- A retail analyst combines twelve monthly sales files into one report with a short script.
- A student renames hundreds of photos automatically instead of doing it by hand.
- A developer builds the back end of a website or mobile app that serves its data.

Who should learn Python?

Python is a good first language for school and college students, career switchers, and working professionals in analytics, finance, engineering or marketing who want to automate routine work and understand data. **No previous programming experience is required.**

> **REMEMBER**
> Python 3 is the current major version. Always install **Python 3**; every example in these notes is written for Python 3.

2 Installing Python and Your First Program

Installing Python

1. Open **python.org**, go to the **Downloads** section and download the latest Python 3 installer for your operating system.
2. Run the installer. On Windows, tick **Add python.exe to PATH** before clicking **Install Now**.
3. Verify the installation: open **Command Prompt** (Windows) or **Terminal** (macOS/Linux) and type the command below.

CODE EXAMPLE
```
python --version
```

OUTPUT
```
Python 3.12.3
```

Your version number may be different. Any version starting with 3 is fine. On macOS and Linux you may need to type `python3 --version`.

Code editors and IDEs

A **code editor** is a program for writing code. An **IDE** (Integrated Development Environment) adds tools such as running, debugging and suggestions in one window.

- **VS Code:** a free, lightweight editor. Install its Python extension to run programs with one click.
- **Jupyter Notebook:** code is written in small blocks called cells and the output appears right below each cell. It is popular for learning and data analysis.

Your first program

SYNTAX
```
print(value)
```

CODE EXAMPLE
```
print("Hello, World!")
```

OUTPUT
```
Hello, World!
```

Part of the code	Meaning
`print`	A built-in **function** that displays something on the screen.
`( )`	Parentheses hold the value you pass to the function.
`"Hello, World!"`	A **string**: text written inside quotation marks.

Real-life use: `print()` is how programs report results, from a simple calculator to a data report. It is also the quickest way to check what your code is doing.

How to run it: save the code in a file named `hello.py`, open a terminal in that folder and type `python hello.py`. In VS Code use the Run button; in Jupyter press Shift + Enter.

COMMON MISTAKES

`Print("Hi")` fails because Python is **case-sensitive**: the function is `print`.
Forgetting a closing quotation mark or bracket, or pasting curly “smart” quotes from a document instead of straight quotes.

3 Syntax, Comments and Indentation

Syntax, statements and case sensitivity

Syntax is the set of rules that decides how code must be written. A **statement** is one complete instruction, and in Python each statement normally sits on its own line. Python is **case-sensitive**: `name`, `Name` and `NAME` are three different names.

CODE EXAMPLE

```
# This is a single-line comment
name = "Asha"      # inline comment
Name = "Ravi"
print(name)
print(Name)
```

OUTPUT

```
Asha
Ravi
```

Comments

A **comment** is a note for humans. Python ignores everything after a `#` symbol on that line. Good comments explain why the code does something, which helps you and others understand it later.

SYNTAX

```
# comment text
```

Indentation

Indentation means the spaces at the start of a line. Many languages use curly braces to group code, but Python uses indentation. Lines that belong together (a **block**) must be indented by the same amount; the standard is **4 spaces**.

CODE EXAMPLE

```
age = 20
if age >= 18:
    print("Eligible to vote")
    print("Please carry your ID")
print("Done")
```

OUTPUT

```
Eligible to vote
Please carry your ID
Done
```

Here the two indented `print` lines belong to the `if` block and run only when the condition is true. The last `print` is not indented, so it always runs.

CODE EXAMPLE

```
age = 20
if age >= 18:
print("Eligible to vote")
```

ERROR

```
IndentationError: expected an
indented block after 'if'
statement on line 2
```

The difference: in the first program the line after `if` is indented, so Python knows it belongs to the block. In the second it is not, so Python cannot tell what to run when the condition is true and stops with an error. The exact wording of the message can vary between Python versions.

Basic naming conventions

Style	Used for	Example
snake_case	Variable and function names (lowercase words joined by underscores)	`student_name, total_marks`
UPPER_CASE	Constants: values that should not change	`PASS_MARK`
PascalCase	Class names (introduced in later notes)	`StudentRecord`

PRO TIP

Choose meaningful names. `total_marks` explains itself; `x` does not.

4 Variables and Data Types

What is a variable?

A **variable** is a named container that stores a value in the computer's memory so you can use it later. Think of a labelled box: the label is the variable name and the item inside is its value. Storing a value is called **assignment**.

SYNTAX
```
variable_name = value
```

Variable naming rules

- Use letters, digits and underscores only.
- Do not start with a digit.
- No spaces or symbols such as `-` or `@`.
- Do not use reserved words such as `if`, `for`, `class`.
- Names are case-sensitive.

Examples

Valid: `student_name`, `age2`, `_total`

Invalid: `2marks` (starts with a digit), `student name` (has a space), `class` (reserved word)

Basic data types

Every value has a **data type** that tells Python what kind of data it is.

Type	Meaning	Example
`int`	Integer: a whole number	`25, -7, 0`
`float`	A number with a decimal point	`86.5, 3.14`
`str`	String: text inside quotes	`"Asha", 'Python'`
`bool`	Boolean: only True or False	`True, False`
`NoneType`	The special value None, meaning "no value yet"	`None`

Practical example: a student record and type()

The built-in function `type()` tells you the data type of a value or variable.

CODE EXAMPLE
```
student_name = "Asha Patel"
age = 19
percentage = 86.5
is_enrolled = True
mentor = None

print(student_name, type(student_name))
print(age, type(age))
print(percentage, type(percentage))
print(is_enrolled, type(is_enrolled))
print(mentor, type(mentor))
```

OUTPUT
```
Asha Patel <class 'str'>
19 <class 'int'>
86.5 <class 'float'>
True <class 'bool'>
None <class 'NoneType'>
```

The difference between = and ==

`=` is the **assignment** operator: it stores a value in a variable. `==` is the **comparison** operator: it checks whether two values are equal and gives `True` or `False`.

CODE EXAMPLE
```
marks = 75
print(marks == 75)
print(marks == 80)
```

OUTPUT
```
True
False
```

REMEMBER
A variable can be given a new value at any time (`age = 19` and later `age = 20`). Python always uses the most recent value.

5 Input and Output

Displaying output with print()

`print()` shows values on the screen. You can print several values separated by commas, join text with `+`, or use an **f-string**: put the letter `f` before the opening quote and place variables inside curly braces `{ }`.

CODE EXAMPLE

```
name = "Riya"
print("Hello", name)
print("Hello, " + name + "!")
print(f"Welcome, {name}!")
```

OUTPUT

```
Hello Riya
Hello, Riya!
Welcome, Riya!
```

Taking input with input()

SYNTAX

```
variable = input("message to show the user")
```

`input()` pauses the program, shows your message and waits for the user to type and press Enter. **Whatever the user types is always returned as a string**, even if it looks like a number. To calculate with it you must convert it.

Type conversion

Function	Converts to	Example	Result
`int()`	Whole number	`int("25")`	`25`
`float()`	Decimal number	`float("85.5")`	`85.5`
`str()`	Text (string)	`str(100)`	`'100'`

Program: student name and marks

Problem: ask the user for a student's name and marks, then display the details. **Sample input:** `Riya Sharma` and `88`.

CODE EXAMPLE

```
name = input("Enter student name: ")
marks = int(input("Enter marks (out of 100): "))

print("----- Student Details -----")
print("Name  :", name)
print("Marks :", marks)
print("Marks after 5 bonus marks:", marks + 5)
```

OUTPUT

```
Enter student name: Riya Sharma
Enter marks (out of 100): 88
----- Student Details -----
Name  : Riya Sharma
Marks : 88
Marks after 5 bonus marks: 93
```

Explanation

- Line 1 stores the typed name as a string. No conversion is needed because a name is text.
- Line 2 wraps `input()` inside `int()`, converting the typed text `"88"` into the number `88`.
- Without `int()`, `marks + 5` would raise a `TypeError`, because text and numbers cannot be added.

Real-life use: registration forms, billing counters and quiz programs all rely on `input()` and `print()`.

6 Python Operators

An **operator** is a symbol that performs an action on values, which are called **operands**. In `10 + 3`, the `+` is the operator and 10 and 3 are the operands. Python has five operator groups that beginners use daily.

Category	Operator	Meaning	Example
Arithmetic	`+ - *`	Add, subtract, multiply	`10 + 3 gives 13`
	`/`	Divide (result is always a float)	`10 / 4 gives 2.5`
	`//`	Floor division (whole-number part)	`10 // 3 gives 3`
	`%`	Modulus (remainder)	`10 % 3 gives 1`
	`**`	Power (exponent)	`2 ** 3 gives 8`
Comparison	`== !=`	Equal to, not equal to	`5 == 5 gives True`
	`> <`	Greater than, less than	`7 > 9 gives False`
	`>= <=`	Greater or equal, less or equal	`5 >= 5 gives True`
Logical	`and`	True only if both sides are true	`True and False gives False`
	`or`	True if at least one side is true	`True or False gives True`
	`not`	Reverses the result	`not True gives False`
Assignment	`=`	Stores a value	`x = 10`
	`+= -=`	Updates the variable in place	`x += 2 means x = x + 2`
Membership	`in`	Is the item present?	`"py" in "python" gives True`
	`not in`	Is the item absent?	`"z" not in "python" gives True`

Arithmetic operators in action

CODE EXAMPLE

```
a = 17
b = 5
print(a + b, a - b, a * b)
print(a / b, a // b, a % b, a ** 2)
```

OUTPUT

```
22 12 85
3.4 3 2 289
```

Comparison, logical, assignment and membership

CODE EXAMPLE

```
marks = 72
print(marks >= 33 and marks <= 100)
print(marks < 33 or marks > 100)
print(not marks > 90)
print("py" in "python")
marks += 10
print(marks)
```

OUTPUT

```
True
False
True
True
82
```

REMEMBER

Division with `/` always returns a **float** (`10 / 5` gives `2.0`). Use `//` for a whole number and `%` to get the remainder. `%` is useful for even or odd checks: a number is even when `number % 2 == 0`.

7 Conditional Statements

Programs often need to make decisions: if the marks are 33 or more, the student passes; otherwise the student fails. **Conditional statements** run different code depending on whether a **condition** is `True` or `False`. Without them a program could only run the same steps every time.

SYNTAX

```
if condition:
    statements              # runs when the condition is True
elif another_condition:
    statements              # checked only if the earlier ones were False
else:
    statements              # runs when nothing above was True
```

`elif` (short for "else if") and `else` are optional. `if` alone is the simplest form.

if and if-else

CODE EXAMPLE

```
number = 7
if number % 2 == 0:
    print(number, "is even")
else:
    print(number, "is odd")
```

OUTPUT

```
7 is odd
```

How it works: `7 % 2` is `1`, so `number % 2 == 0` is `False`. Python skips the `if` block and runs the `else` block.

if-elif-else: checking several conditions

CODE EXAMPLE

```
marks = 62
if marks >= 75:
    print("Distinction")
elif marks >= 33:
    print("Pass")
else:
    print("Fail")
```

OUTPUT

```
Pass
```

How it works: Python checks the conditions from top to bottom and runs only the **first** one that is true. `62 >= 75` is false, `62 >= 33` is true, so it prints `Pass` and ignores the rest.

Nested if

An `if` placed inside another `if` is called a **nested if**. The inner check runs only when the outer condition is true.

CODE EXAMPLE

```
age = 20
has_ticket = True
if age >= 18:
    if has_ticket:
        print("Entry allowed")
    else:
        print("Please buy a ticket")
```

OUTPUT

```
Entry allowed
```

REAL-LIFE USE

Login checks, discount rules, ATM menus, grading systems and traffic-light logic are all built from conditions.

8 Loops in Python

A **loop** repeats a block of code so you do not have to write the same lines again and again. Python has two loops: `for`, which repeats over a sequence of values, and `while`, which repeats as long as a condition stays true.

SYNTAX

```
for variable in sequence:
    statements

while condition:
    statements
```

The for loop and range()

`range()` produces a sequence of numbers. The stop value is **not** included.

- `range(5)` gives 0, 1, 2, 3, 4 | `range(1, 6)` gives 1, 2, 3, 4, 5
- `range(1, 10, 2)` gives 1, 3, 5, 7, 9 (the third value is the **step**)

CODE EXAMPLE

```
for number in range(1, 6):
    print(number)
```

OUTPUT

```
1
2
3
4
5
```

CODE EXAMPLE

```
total = 0
for number in range(1, 6):
    total = total + number
print("Sum =", total)
```

OUTPUT

```
Sum = 15
```

The while loop

CODE EXAMPLE

```
countdown = 3
while countdown > 0:
    print(countdown)
    countdown -= 1
print("Liftoff!")
```

OUTPUT

```
3
2
1
Liftoff!
```

The condition is checked before every repeat. Each pass reduces `countdown` by 1, so after three passes the condition becomes false and the loop ends.

break and continue

CODE EXAMPLE

```
for number in range(1, 8):
    if number == 3:
        continue
    if number == 6:
        break
    print(number)
```

OUTPUT

```
1
2
4
5
```

`continue` skips the rest of the current pass (so 3 is not printed). `break` ends the loop completely (so the loop stops at 6).

	for loop	while loop
Best when	You know how many times to repeat	You repeat until something changes
Controlled by	A sequence such as range(1, 6)	A condition such as count > 0
Main risk	Rarely runs forever	Runs forever if the condition never becomes false

Tip: if a `while` loop never stops, press **Ctrl + C** in the terminal and check that something inside the loop changes the condition.

9 Strings and Basic String Operations

A **string** is a sequence of characters, such as letters, digits, spaces and symbols, written inside single or double quotation marks (for example `"PYTHON"` or `'PYTHON'`). Strings store names, addresses, messages and any other text.

Indexing and slicing

Each character has a position called an **index**, starting at **0**. Negative indexes count from the end. **Slicing** extracts a part of the string: `text[start:stop]` includes `start` but **excludes** `stop`.

Character	P	Y	T	H	O	N
Index	0	1	2	3	4	5
Negative index	-6	-5	-4	-3	-2	-1

CODE EXAMPLE

```
word = "PYTHON"
print(word[0])
print(word[-1])
print(word[0:3])
print(word[2:])
```

OUTPUT

```
P
N
PYT
THON
```

Joining, repeating and measuring

CODE EXAMPLE

```
first = "Riya"
last = "Sharma"
full_name = first + " " + last
print(full_name)
print("-" * 10)
print(len(full_name))
```

OUTPUT

```
Riya Sharma
----------
11
```

`+` joins (**concatenates**) strings, `*` repeats a string, and `len()` counts the characters, including spaces.

Common string methods

A **method** is a function that belongs to a value and is called with a dot. Methods return a **new** string; the original is not changed.

Method	What it does	Example	Result
`upper()`	Converts to capital letters	`"python".upper()`	`'PYTHON'`
`lower()`	Converts to small letters	`"PYTHON".lower()`	`'python'`
`strip()`	Removes spaces at both ends	`"  hi  ".strip()`	`'hi'`
`replace()`	Replaces one piece of text with another	`"banana".replace("a", "o")`	`'bonono'`
`split()`	Breaks text into a list of words	`"red,green".split(",")`	`['red', 'green']`

Practical example: cleaning a student name

CODE EXAMPLE

```
raw_name = "  riya sharma  "
clean_name = raw_name.strip().upper()
print(clean_name)
words = clean_name.split()
print(words)
print("First name:", words[0])
```

OUTPUT

```
RIYA SHARMA
['RIYA', 'SHARMA']
First name: RIYA
```

10 Lists and Tuples

Lists

A **list** stores several values in one variable. It is **ordered** (items keep their position) and **changeable** (you can add, update or remove items). Lists are written in square brackets, and the first item has index 0.

CODE EXAMPLE

```
subjects = ["Maths", "Science", "English"]
print(subjects[0])
subjects[1] = "Physics"
subjects.append("History")
print(subjects)
print(len(subjects))
```

OUTPUT

```
Maths
['Maths', 'Physics', 'English', 'History']
4
```

`subjects[0]` reads the first item, `subjects[1] = "Physics"` updates the second item, `append()` adds an item at the end and `len()` counts the items.

Removing items and looping

CODE EXAMPLE

```
subjects.remove("English")
removed = subjects.pop()
print(subjects, removed)
for subject in subjects:
    print(subject)
```

OUTPUT

```
['Maths', 'Physics'] History
Maths
Physics
```

`remove("English")` deletes an item by **value**. `pop()` removes the **last** item and gives it back (use `pop(0)` to remove by position). The `for` loop visits each remaining item one by one.

Tuples

A **tuple** is like a list, but it is **unchangeable** once created. Tuples are written in round brackets and are used for data that should stay fixed, such as coordinates or days of the week.

CODE EXAMPLE

```
coordinates = (10, 20)
print(coordinates[0])
print(len(coordinates))
```

OUTPUT

```
10
2
```

CODE EXAMPLE

```
coordinates[0] = 5
```

ERROR

```
TypeError: 'tuple' object does not
support item assignment
```

Lists compared with tuples

Feature	List	Tuple
Brackets	`[ ]` square	`( )` round
Can items be changed?	Yes (changeable)	No (unchangeable)
Adding and removing	append(), remove(), pop()	Not available
Best used for	Data that changes, such as marks or a shopping cart	Fixed data, such as coordinates or days of the week

11 Functions and Basic Error Handling

Functions

A **function** is a named, reusable block of code that performs one task. Functions save you from repeating code, make programs shorter and easier to fix, and let you build large programs from small pieces.

SYNTAX

```
def function_name(parameters):
    statements
    return value
```

- **Parameter:** a variable in the function definition that receives a value.
- **Argument:** the actual value you pass when you call the function.
- **return:** sends a result back to the place where the function was called.

CODE EXAMPLE

```
def square(number):
    return number * number

result = square(5)
print(result)
print(square(9))
```

OUTPUT

```
25
81
```

`def` creates the function, `number` is the parameter, and `square(5)` calls it with the argument `5`. The returned value 25 is stored in `result`.

Common errors

Errors are a normal part of programming. Python's error message tells you the **type** of error and usually the line where it happened. Read it carefully; it often explains the fix.

Error	What it means	Example and message
`SyntaxError`	The code breaks the rules of the language	`print("Hi"` Message: '(' was never closed
`NameError`	A name is used before it has been created	`print(score)` Message: name 'score' is not defined
`TypeError`	An operation is used on the wrong type of data	`"5" + 3` Message: can only concatenate str (not "int") to str
`IndentationError`	The spaces at the start of a line are wrong	`if True:` `print("x")` (not indented) Message: expected an indented block

Handling errors with try and except

An error that stops a program while it runs is called an **exception**. With `try` and `except` you can catch it and respond politely instead of crashing.

CODE EXAMPLE

```
try:
    number = int("abc")
    print(number)
except ValueError:
    print("Please enter a valid number.")
```

OUTPUT

```
Please enter a valid number.
```

Python tries the `try` block first. `int("abc")` fails with a `ValueError`, so Python jumps to the `except` block. If no error occurs, the `except` block is skipped.

12 Mini Project: Student Marks Calculator

Objective

Build a program that accepts a student's name and marks in three subjects, then displays the total, the percentage and the result.

Concepts used

Variables, `input()`, `int()`, arithmetic operators, `if-else`, logical operator `and`, and `print()`.

> **REMEMBER**
> **Passing rule:** each subject is out of 100. A student passes only if the marks in **every** subject are at least 33. Otherwise the result is FAIL.

CODE EXAMPLE

```
# Student Marks Calculator
name = input("Enter student name: ")
subject1 = int(input("Enter marks in Subject 1: "))
subject2 = int(input("Enter marks in Subject 2: "))
subject3 = int(input("Enter marks in Subject 3: "))

total = subject1 + subject2 + subject3
percentage = total / 300 * 100

if subject1 >= 33 and subject2 >= 33 and subject3 >= 33:
    result = "PASS"
else:
    result = "FAIL"

print()
print("------ Result ------")
print("Name       :", name)
print("Total      :", total, "out of 300")
print("Percentage :", round(percentage, 2), "%")
print("Result     :", result)
```

SAMPLE INPUT AND EXPECTED OUTPUT

```
Enter student name: Riya Sharma
Enter marks in Subject 1: 78
Enter marks in Subject 2: 85
Enter marks in Subject 3: 69

------ Result ------
Name       : Riya Sharma
Total      : 232 out of 300
Percentage : 77.33 %
Result     : PASS
```

Step-by-step explanation

1. `input()` collects the name; `int()` turns each typed mark into a number.
2. `total` adds the marks (maximum 300). `percentage` is `total / 300 * 100`, and `round(percentage, 2)` shows two decimal places.
3. The `if` line uses `and`, so all three conditions must be true. One mark below 33 makes the result FAIL.
4. The final `print()` lines display the name, total, percentage and result.

Try this: enter marks `45`, `30` and `60`. The total is 135 and the percentage is 45.0, but the result is **FAIL** because one subject is below 33.

13 Practice Questions and Learning Roadmap

Try these questions on your own. Write the code, run it and compare the result with what you expected. Solving them without looking at the notes is the best way to learn.

No.	Type	Question
1	Concept	What is the difference between `=` and `==`?
2	Concept	Why is indentation important in Python?
3	Concept	Give two differences between a list and a tuple.
4	Output prediction	What does `print(10 // 3, 10 % 3)` display?
5	Output prediction	What is the output of `word = "PYTHON"` followed by `print(word[1:4])`?
6	Output prediction	What numbers does `for i in range(2, 10, 3): print(i)` print?
7	Programming	Ask for a name with `input()` and print a welcome message that includes it.
8	Programming	Read a number and report whether it is positive, negative or zero.
9	Programming	Read three numbers and print the largest of them.
10	Programming	Print the multiplication table of a number entered by the user (1 to 10).

Bonus challenges: create a simple calculator, count the characters in a string, find the sum of a list, and write a function `add_numbers(a, b)` that returns the sum of two numbers.

WHAT'S NEXT?

The Python fundamentals in this sample (variables, conditions, loops, strings, lists and functions) are the building blocks of every topic below. Each upcoming area builds directly on them.

Step	Learning area	What it adds
1	**Python for Data Analytics**	Apply Python to real datasets and answer questions with data
2	**NumPy and Pandas**	Work efficiently with arrays and tables of data
3	**Data Visualization**	Turn data into clear charts and graphs
4	**Advanced Excel**	Analyse and report on data with powerful spreadsheet tools
5	**SQL and Database Management**	Store, query and manage data in databases
6	**Power BI and Business Intelligence**	Build interactive dashboards and business reports
7	**Machine Learning and Artificial Intelligence**	Build systems that find patterns and make predictions

CONTINUE YOUR LEARNING JOURNEY

Want to continue your learning journey? Explore our complete educational notes covering Python, Data Analytics, Advanced Excel, SQL, Power BI, and more.

Visit **www.pinakiithub.com** for available learning resources.

Final page of the sample learning material | Thank you for reading